

LSMTree Benchmark Setup Guide

Sep 27, 2026 · @Sarvesh

Purpose and ground rules

Every performance number in your PRs should come from this one harness, run in the same environment as LevelDB and RocksDB. Absolute numbers change from machine to machine; the ratio to the reference engines on your machine is the number that matters.

- **Always run all three engines together.** LSMTree, LevelDB 1.23 and RocksDB 8.9 run the identical workload in the same environment, in the same session.
- **Don't change the workload.** The workload contract below is fixed. Tune your own engine however you like, but document every setting you change.
- **Don't handicap the reference engines.** Use the settings in the contract, nothing weaker.
- **Wrong answers don't count.** The harness checks every read and exits non-zero if a result is wrong. A fast engine that returns wrong data is a failed run.
- **Benchmark on Linux.** Native, VM or Docker all work. macOS gives misleading fsync numbers (see Environment).
- **Profile before you optimize.** Each PR should show where the time goes, before and after, not just the headline numbers.

What's in the starter kit

The kit is a folder called `kvbench` . Put it at `bench/kvbench/` in your repo and commit it, so the harness version travels with your PRs.

Path	Purpose	Edit it?
<code>cpp/kvbench.cc</code>	Runs the workload against LevelDB and RocksDB, plus a raw disk-sync probe	No
<code>java/LSMTree/KvBench.java</code>	Runs the same workload against LSMTree	No (workload is fixed)

Path	Purpose	Edit it?
<code>java/LSMTree/CandidateAdapter.java</code>	Connects the harness to your engine's API	Yes, as your API changes
<code>scripts/run_all.sh</code>	Builds everything, runs all engines 3 times, writes a markdown summary	Only env variables
<code>scripts/syscalls_per_op.sh</code>	Counts I/O syscalls per get, miss and scan for each engine	Only env variables
<code>scripts/profile.sh</code>	Flame graph of one phase of your engine, via async-profiler	Only env variables
<code>scripts/summarize.py</code>	Turns raw results into the summary tables	No
<code>Dockerfile</code>	Ubuntu 24.04 with JDK 25, LevelDB 1.23, RocksDB 8.9.1, strace	No
<code>README.md</code>	Command quick reference	No

The C++ and Java harnesses share one random-number generator (SplitMix64), one seed and one key/value generator. All three engines see exactly the same keys, in the same order.

Environment setup

Run on Linux, pin the run to one CPU core, and keep the data directory on a real Linux filesystem. Docker satisfies all three on any host.

Option A: Docker (any host, recommended). From the repo root:

```
docker build -t lsm-bench bench/kvbench
docker run --rm -it --cpuset-cpus=0 --cap-add=SYS_PTRACE \
  -v "$PWD":/repo -v lsm-bench-data:/data -e DATA_DIR=/data/kvbench -w
/repo lsm-bench \
  bench/kvbench/scripts/run_all.sh
```

- The database files go on the named volume `/data` , never on the bind-mounted repo. On macOS and Windows, bind mounts pass through a file-sharing layer with very different `fsync` and read behavior.
- `--cap-add=SYS_PTRACE` is only needed for `syscalls_per_op.sh` .

Option B: native Linux. On Ubuntu 24.04:

```
sudo apt-get install -y openjdk-25-jdk-headless g++ librocksdb-dev
libleveldb-dev python3 strace
CPUS=0 bench/kvbench/scripts/run_all.sh
```

On other distributions you need JDK 22 or newer (the engine uses unnamed `_` variables), `g++` with `C++20`, the LevelDB and RocksDB development packages, `python3` and `strace`.

Why not macOS directly. On macOS, a plain `fsync()` does not force the drive to write its cache to stable storage; that needs `fcntl(F_FULLFSYNC)` . Durable-write numbers measured natively on a Mac can therefore look several times better than real. Docker Desktop runs a Linux VM, so relative comparisons inside it are valid. Absolute sync numbers there may still be optimistic, which is why every run also reports the machine's sync floor.

Pinning and noise. The reference numbers below come from a 1-vCPU machine, so pin to one core (`CPUS=0` natively, `--cpuset-cpus=0` in Docker). Plug laptops in and close heavy apps. Never benchmark while a build or another benchmark is running. If you also run on more cores to study concurrency, report those numbers separately and say so.

Component	Version used for reference numbers
OS / kernel	Ubuntu 24.04, ext4
JDK	OpenJDK 25.0.4 (22+ required)
LevelDB	1.23 (Ubuntu package <code>libleveldb-dev</code>)
RocksDB	8.9.1 (Ubuntu package <code>librocksdb-dev</code>)
Compiler	<code>g++ 13.3</code>

Quickstart

One command builds everything and produces a results table for all three engines. It takes about 8 minutes on one core.

1. Copy the kit to `bench/kvbench/` in your repo and commit it.
2. Set up the environment (Docker image or native packages, above).
3. Smoke-test the setup with a small run. These numbers are only a setup check; never report them.

```
RUNS=1 EXTRA_ARGS="--keys 20000 --hits 20000 --misses 20000 --scans 2000 -
-warmup 2000 --per-writer 500" \
  bench/kvbench/scripts/run_all.sh
```

4. Run the real benchmark: `bench/kvbench/scripts/run_all.sh` (3 runs by default).
5. Open `bench/kvbench/results/<timestamp>/summary.md`. Next to it are `raw.txt` (every result line), `env.txt` (machine details) and one log per engine per run.

If any engine returns a wrong answer, the script stops, prints the `VERIFY_FAILED` line and names the log file. Fix correctness first; a run that fails verification has no valid numbers.

Variable	Default	Use
<code>RUNS</code>	3	Number of full runs; the summary reports medians
<code>CPUS</code>	unset	Core list for <code>taskset</code> , e.g. <code>0</code> (native Linux only)
<code>DATA_DIR</code>	<code>/tmp/kvbench-data</code>	Where database files go; must be a real Linux filesystem
<code>ENGINES</code>	<code>candidate leveldb rocksdb</code>	Engines to run
<code>SRC</code>	<code>src/main/java</code> or <code>src</code>	Your engine's source root
<code>JAVA_OPTS</code>	<code>-Xms1g -Xmx1g -XX:+AlwaysPreTouch</code>	JVM flags; list any change in your PR
<code>EXTRA_ARGS</code>	none	Harness flags for smoke runs only

The workload contract

This workload is fixed for the whole exercise, so every number stays comparable across your PRs and against the reference engines. It has the same shape LevelDB's own `db_bench` uses: 16-byte keys and 100-byte values.

Data. 100,000 keys of the form `user` + 12-digit number (16 bytes). Each value is 100 bytes: the key itself followed by 84 deterministic filler bytes. Keys are inserted in a random order from SplitMix64 with seed 42. LevelDB and RocksDB run with compression off and 10-bit-per-key bloom filters; everything else is default.

Phases, in order. Phases run single-threaded unless noted. Every write phase starts from a fresh, empty database.

#	Phase	What happens	Reported	Verified
1	<code>fsync_floor</code>	2,000 appends of 120 bytes, each followed by <code>fsync</code> (then again with <code>fdatasync</code>)	ops/sec, latency	—
2	<code>fill_nosync</code>	100,000 non-durable puts	ops/sec, latency	— ("unsupported" allowed)
3	<code>fill_sync</code>	100,000 durable puts; this database is used by phases 4–7	ops/sec, latency	—
4	<code>settle</code>	Flush the memtable, finish background work, compact fully	seconds, disk MiB	—
5	<code>read_hit</code>	20,000 warm-up gets, then 100,000 gets of random existing keys	ops/sec, latency	Value matches byte for byte
6	<code>read_miss</code>	100,000 gets of keys that sort between real keys (<code><key>x</code>)	ops/sec, latency	Must return "absent"
7	<code>scan100</code>	1,000 warm-up scans, then 10,000 scans of 100 consecutive keys from a random start	scans/sec, latency	100 rows, ascending, starting at the start key, each value begins with its own key

#	Phase	What happens	Reported	Verified
8	<code>fill_sync_mt</code>	8 threads, each doing 5,000 durable puts of distinct keys	total ops/sec	Every write readable afterward

What "durable" and "non-durable" mean. A durable put must survive power loss once it returns. A non-durable put must survive a process crash (the data has been handed to the OS) but may be lost if the OS crashes or power fails. That matches LevelDB and RocksDB with `sync=false`. Holding writes only in your own process's buffers is weaker than that, and doesn't count as non-durable mode.

Rules.

- Don't change counts, key or value shapes, the seed, the phase order, the verification or the settle definition.
- The adapter only translates calls. It must not cache reads, batch writes, skip work or do anything else your engine wouldn't do for a real caller.
- If a phase needs a feature your engine doesn't have yet, throw `UnsupportedOperationException`; the summary shows "unsupported".

Adapting the harness as your code changes

`CandidateAdapter.java` is the only file you should edit. It implements this interface from `KvBench.java`:

```
public interface Kv extends AutoCloseable {
    void put(String key, byte[] value, boolean sync) throws IOException; //
    sync=true: durable on return
    byte[] get(String key) throws IOException;                          //
    null if absent or deleted
    int scan(String low, String high, int limit, Visitor v) throws
    IOException; // [low, high], ascending
    void settle() throws IOException; // memtable flushed, background work
    done, fully compacted
    String describe(); // your engine config, printed next
    to the results
    void close() throws IOException;
}
```

The starting adapter mirrors LevelDB's defaults: a 4 MiB memtable, a sparse-index entry every 16 keys, and leveled compaction with 2 MiB target files. `settle()` flushes, waits for compaction, runs one full compaction and waits again. It relies on package-private methods (`flush`, `awaitCompaction`, `sharedCompactionLogic`, `segments`, `scan`), which is the only reason the harness lives in package `LSMTree`.

When your API changes, adapt as follows:

- **New public methods** (for example a public `scan`, `delete` or `compactRange`): switch the adapter to them and drop the package-private calls.
- **A non-durable write option:** forward the `sync` flag instead of throwing, and `fill_nosync` starts reporting.
- **Moving to Gradle or Maven:** the scripts auto-detect `src/main/java`; set `SRC` if your layout differs, and `JAVA_CP_EXTRA` if the engine gains dependencies.
- **Renaming packages:** move `KvBench.java` and `CandidateAdapter.java` into your new package; change nothing else in `KvBench.java`.
- **Changing any engine setting** (memtable size, block size, cache size, compaction parameters): update `describe()` so the setting is printed with every result.

Methodology: numbers you can trust

Report the median of 3 full runs, next to the reference engines from the same session. Treat a difference under about 10% as noise unless it reproduces.

- **Runs and order.** `run_all.sh` runs every engine 3 times and rotates the engine order each run, so no engine always goes first. The summary shows medians with the [min-max] range.
- **Noise.** If two results' min-max ranges overlap, you haven't shown a difference. Re-run with `RUNS=5` before claiming a small win or dismissing a small regression.
- **Warm-up.** The JVM compiles hot code while it runs, so both harnesses do 20,000 untimed gets and 1,000 untimed scans before each read phase. Don't add or remove warm-up.
- **Settle before reads.** Reads run after a flush and full compaction, so they measure the steady state, not a half-compacted tree. The summary also reports settle time and disk usage, which shows your space amplification.
- **Working set.** The dataset is about 11 MiB, so it fits in memory. These are in-memory numbers, the same situation as LevelDB's published benchmarks. Don't drop the OS page cache between phases. If you study larger-than-memory behavior, run it

separately with a bigger `--keys` and a container memory limit, and label it clearly.

- **Sync floor.** Each run starts by timing single 120-byte appends followed by `fsync`. That is the cost of one durable append on this machine; it is reported at the bottom of the summary.
- **Latency, not just throughput.** Every single-threaded op is timed individually, giving p50, p99 and max. A large max usually means a stall: a flush or compaction on the request path. Report p99 next to throughput.
- **Measurement cost is shared.** Timer calls and the built-in correctness checks add a few tens of nanoseconds per op, identically in both harnesses.
- **Clean runs only.** Never report numbers from a smoke run, a profiled run, a run under `strace`, or a build with debug logging or extra assertions.

Profiling: finding where the time goes

Find the cause before you change code. The kit gives you two views: a flame graph of one phase, and a count of I/O syscalls per operation.

Flame graphs. `profile.sh` downloads `async-profiler` 4.5 on first use. It attaches to the JVM for exactly one phase's timed loop, so the load phase doesn't drown out the reads. Output is an HTML flame graph under `results/`.

```
PHASE=read_hit EVENT=cpu bench/kvbench/scripts/profile.sh # where CPU time
goes
PHASE=read_hit EVENT=wall bench/kvbench/scripts/profile.sh # wall time,
including time blocked in syscalls
PHASE=scan100 EVENT=alloc bench/kvbench/scripts/profile.sh # allocation hot
spots (GC pressure)
PHASE=fill_sync_mt EVENT=lock bench/kvbench/scripts/profile.sh # contention
between writer threads
```

- `PHASE` can be `fill_sync`, `read_hit`, `read_miss`, `scan100` or `fill_sync_mt`.
- For I/O-heavy phases, `wall` is usually more revealing than `cpu`: `cpu` only shows time spent running, not time spent waiting in the kernel. Stacks are split per thread, so look at the `main` thread for single-threaded phases.
- If `cpu` fails inside Docker because `perf` events are blocked, use `EVENT=itimer`.

Syscalls per operation. `syscalls_per_op.sh` runs each engine under `strace`: once for load and settle only, then once more with 2,000 extra gets, misses or scans. The difference divided by 2,000 is the number of I/O syscalls per operation, with the top

syscalls listed. It takes about 5 minutes; in Docker it needs `--cap-add=SYS_PTRACE`. Compare your engine with the reference numbers below: a large count per get is a strong clue.

Java Flight Recorder (no install). Add `-`

`XX:StartFlightRecording=filename=rec.jfr,settings=profile` to `JAVA_OPTS` for one run. Open `rec.jfr` in JDK Mission Control, or convert it with `async-profiler's` `jfrconv` `rec.jfr` `flame.html`. It records the whole run, load phase included.

Microbenchmarks for components. To optimize one component in isolation (bloom filter, block decoding, key encoding, memtable insert), use [JMH](#), not a hand-written timing loop. Hand-written loops under about 1 μ s per op are distorted by JIT warm-up and dead-code elimination. Always confirm a component win on the end-to-end harness.

Reporting results in your PRs

Every PR that claims a performance change needs before and after numbers from the same session, on the same machine, with the reference engines alongside.

Commit the kit to `main` first. The easiest way to get both numbers from one session is then a second worktree for `main`, running the kit from each checkout back to back:

```
git worktree add ../lsm-main main
(cd ../lsm-main && CPUS=0 bench/kvbench/scripts/run_all.sh) # before
CPUS=0 bench/kvbench/scripts/run_all.sh                    # after (your
branch)
```

Then paste this into the PR description:

Performance

```

**Mechanism:** <one line: what this PR changes>
**Evidence of the problem:** <flame graph or syscalls-per-op numbers from
main, attached>
**Environment:** <paste env.txt>
**Engine settings changed:** <from describe(), e.g. blockSize=4KiB
blockCache=8MiB>

| Workload | main @<sha> | this PR @<sha> | Change | LevelDB | RocksDB |
|---|---|---|---|---|---|
| Point read, key present | | | | | |
| ... one row per workload in the summary ... | | | | | |

**p99 latency ( $\mu$ s), phases this PR touches:** before → after
**Syscalls per op (if I/O changed):** before → after
**Regressions and costs:** <anything slower, more memory, more disk, and why
it's acceptable>
**Durability impact:** <none, or exactly what changed>
**Reproduce:** git checkout <sha> && CPUS=0 bench/kvbench/scripts/run_all.sh

```

Attach flame graphs as a zip, or commit them under `bench/kvbench/results/<pr-name>/`. A regression you found and explained counts in your favor; one we find for you does not.

Reference numbers and targets

These are the reviewer's results with this exact kit, on LSMTree commit `c5282c1`. Use them to check that your setup behaves sensibly, then work from your own machine's numbers.

Machine: a 1-vCPU Intel Xeon (2.1 GHz) VM with 4 GB RAM and ext4, pinned to core 0. Medians of 3 runs, 27 September 2026.

Workload (ops/sec)	LSMTree @c5282c1	LevelDB 1.23	RocksDB 8.9.1	Suggested target for LSMTree
Random writes, fsync each (1 thread)	9,004	9,389	8,773	report it

Workload (ops/sec)	LSMTree @c5282c1	LevelDB 1.23	RocksDB 8.9.1	Suggested target for LSMTree
Random writes, fsync each (8 threads)	8,451	39,492	29,998	≥ 25,000
Random writes, no per- write fsync	unsupported	554,571	199,766	≥ 200,000
Point read, key present	15,780	625,247	386,608	≥ 150,000
Point read, key absent	219,622	1,741,455	2,028,480	≥ 500,000
Range scan, 100 keys (scans/sec)	769	79,473	30,395	≥ 15,000

Other numbers from the same run:

- **p99 latency, point read (key present):** LSMTree 141 μ s, LevelDB 2.1 μ s, RocksDB 4.2 μ s.
- **After settle:** LSMTree used 12.75 MiB on disk (1.15× the 11.06 MiB of raw data); LevelDB and RocksDB each used about 11.1 MiB (1.00×).
- **Disk sync floor:** 9,939 appends+fsync per second (p50 93 μ s).

I/O syscalls per operation for the reference engines (`syscalls_per_op.sh` , 1,000 ops per phase). Run it for your own engine and compare:

Engine	Per get (present)	Per get (absent)	Per 100-key scan
LevelDB 1.23	0.0	0.0	0.0
RocksDB 8.9.1	0.9	0.0	3.1

The targets are estimates of what a well-built Java engine can reach on this machine and workload. Numbers are a means, not the goal: a smaller gain you can explain fully, with no loss of correctness or durability, beats a bigger one you can't.

Pitfalls and FAQ

My numbers are nowhere near the reference table. Expected: different hardware. Compare the ratios to LevelDB and RocksDB on your machine, not absolute numbers.

LevelDB beats RocksDB here. Is the harness broken? No. On a small, single-threaded, 1-core run with default settings, RocksDB's extra machinery costs more than it saves. Its advantages show up at larger scale and higher concurrency.

LevelDB shows zero syscalls per read. On 64-bit Linux, LevelDB memory-maps its table files, so reads are memory accesses (and page faults), not syscalls.

Can I tune LevelDB or RocksDB? Not in the main table. A tuned comparison is fine as a separate, clearly labeled experiment.

Can I change the JVM heap or garbage collector? Yes, through `JAVA_OPTS`, stated in the PR. Keep it realistic: a huge heap that hides memory pressure isn't a fair result.

The harness passes but my unit tests fail. The harness only checks what it reads back. It is not a correctness suite; your tests still need to pass.

taskset fails in Docker. `CPUS` only works on native Linux. In Docker, use `--cpuset-cpus`. `strace` says "Operation not permitted". Add `--cap-add=SYS_PTRACE` to `docker run`.

The build says JDK 22+ is needed. Install JDK 25 (the Docker image already has it).

Apple Silicon. The Docker image builds for arm64 and `profile.sh` downloads the arm64 profiler. The reference numbers below are from x86-64, so compare ratios.

The results folder keeps growing. Delete old result folders freely; keep only the ones your PRs cite.